Detailed Design Document (Low-Level Design / LLD)

Project: Universal Optical Character Recognition System (`ocr_detection`)

Author: Tech Lead Subagent • **Status:** Approved Engineering Design
Purpose: This Detailed Design Document defines the internal data flow, component sequence, module-level function algorithms, input/output schemas, error-handling mechanisms, and automated test design for the primary code files before implementation begins.

---

1. Core Code Architecture (The Primary Files)

The codebase is strictly structured across modular components:

```
ocr_detection/
├── requirements.txt            <-- Package dependencies
├── packages.txt                <-- Linux graphics dependencies (libgl1,
libglib2.0-0)
│
├── src/
│   ├── utils.py                <-- Module 1: Pre-processing, deskew, and
export tools
│   ├── ocr_engine.py           <-- Module 2: Core AI engine (CRAFT detection
+ CRNN recognition)
│   └── app.py                  <-- Module 3: Streamlit interactive web
dashboard
│
├── tests/
│   ├── test_images/            <-- Curated benchmark photos (plates,
receipts, documents)
│   ├── test_utils.py           <-- Module 4A: 17 Unit tests for pre-
processing & utilities
│   └── test_ocr_engine.py      <-- Module 4B: 9 Integration tests for AI OCR
accuracy
│
├── output/                     <-- Generated visual verification artifacts
│   ├── edge_cases/             <-- Visual outputs for noise, glare, blank,
inverted
│   └── diverse_tests/          <-- Visual outputs for truck plates,
prescriptions, labels
│
└── reports/
    └── security-audit.md       <-- Defensive security review (SAFE TO
COMMIT)
```

---

2. End-to-End System Sequence & Data Flow

The diagram below illustrates the exact runtime flow across all modules:



```
     |                        |       {annotated_image, total_text_blocks, ...}
|
     |                        |
|
     |                        |--- 7. render_metrics(total, confidence,
latency)|
     |                        |--- 8. format_export_data(detections, "csv")
|
     |<-- 9. Display image -|
|
     |    & export buttons  |
|
```

---

3. Module-by-Module Detailed Specifications

Module 1: `src/utils.py` (Validation, Enhancement & Drawing Engine)

1.1 `validate_image_bytes(file_bytes: bytes) -> bool`

- **Purpose**: Verifies that uploaded raw bytes form a valid, uncorrupt image file and protects against 200MB Denial-of-Service attacks.
- **Algorithm**: Checks byte length > 0 and ≤ 200 MB, applies two-phase Pillow verification (`Image.open().verify()` and `load()`), and catches all exceptions to return `False` safely without crashing.

1.2 `preprocess_document_image(image: np.ndarray) -> np.ndarray`

- **Purpose**: Normalizes lighting, eliminates vehicle headlight glare, removes paper shadows, and rotates skewed documents horizontally.
- **Algorithm**:
  1. **CLAHE**: Converts image to LAB color space, applies CLAHE (`clipLimit=2.0`, `tileGridSize=(8, 8)`) on L-channel to balance contrast, merges back to BGR.
  2. **Bilateral Filtering**: Denoises background paper grain while preserving sharp character edge gradients (`d=5`, `sigmaColor=50`, `sigmaSpace=50`).
  3. **Auto-Deskew**: Calculates contour minAreaRect tilt angle and counter-rotates image via affine matrix with white padding.

1.3 `draw_ocr_bounding_boxes(image: np.ndarray, detections: list, box_color: tuple) -> np.ndarray`

- **Purpose**: Draws high-contrast bounding boxes around detected text blocks with adaptive background banners displaying text snippets and confidence percentages.
- **Boundary Clamping Rule**: Clamps badge coordinates (`max(0, min(dim, val))`) so text banners are never cut off at image margins.

1.4 `format_export_data(detections: list, format_type: str = "txt") -> str`

- **Purpose**: Serializes extracted detections into plain text (`.txt`), CSV spreadsheet table (`.csv`), or structured JSON string (`.json`).

Module 2: `src/ocr_engine.py` (Core AI Deep Learning OCR Engine)

2.1 `load_ocr_engine(languages: list = None) -> OCREngineWrapper`

- **Purpose**: Initializes and caches the Deep Learning CRAFT text detector and CRNN character recognizer into memory.
- **Hardware Detection**: Automatically runs on NVIDIA CUDA GPU if available; otherwise runs smoothly on CPU.

2.2 `disambiguate_license_plate(text: str) -> str`

- **Purpose**: Solves classic alphanumeric OCR confusion on vehicle license plates (e.g. `0` vs `O`, `1` vs `I`, `8` vs `B`).
- **Algorithm**: Matches tokens against national vehicle registration schemas (e.g., Indian HSRP: 2 State letters + 2 District digits + 1-2 Series letters + 4 Unique digits). Converts optical numbers to letters in letter slots and letters to digits in numeric slots.

2.3 `detect_and_extract_text(image: np.ndarray, engine: OCREngineWrapper = None, confidence_threshold: float = 0.30) -> dict`

- **Purpose**: The Master Coordinator Function that processes a single image frame end-to-end and packages all results into a clean dictionary.

A. What Goes In (Inputs): * `image: np.ndarray` → The uploaded photo represented as a pixel matrix. * `confidence_threshold: float = 0.30` → Sensitivity filter (e.g., 30% confidence threshold).

B. What Happens Inside (4-Step Sequential Pipeline): * **Step 1 (Preprocess & Deskew):** Invokes `preprocess_document_image()` to normalize contrast, denoise, and straighten tilt. * **Step 2 (Localize & Recognize Characters):** Runs CRAFT text detector to locate character regions, then runs CRNN deep sequence model to read words. * **Step 3 (Disambiguate & Sort):** Applies `disambiguate_license_plate()` to correct vehicle plate numbers, and sorts text blocks in natural reading order (top-to-bottom, left-to-right). * **Step 4 (Paint Annotations):** Calls `draw_ocr_bounding_boxes()` from `src/utils.py` to paint high-contrast bounding boxes and badges on the canvas.

C. What Comes Out (Output Dictionary Schema):

```
{
    # 1. The image array with the bounding boxes and text badges painted on
it
    "annotated_image": np.ndarray,
    # 2. Total number of text blocks found in the photo (e.g., 6)
    "total_text_blocks": 6,
    # 3. Complete extracted plain text string
```

```
    "extracted_text": "PURCHASE ORDER #89201\nVendor: Global Logistical
Supplies\n...",
    # 4. Detailed line-by-line detections list
    "detections": [
        {
            "line_id": 1,
            "text": "PURCHASE ORDER #89201",
            "confidence": 0.892,
            "box": [[40, 70], [450, 70], [450, 110], [40, 110]]
        },
        {
            "line_id": 2,
            "text": "Vendor: Global Logistical Supplies",
            "confidence": 0.704,
            "box": [[40, 120], [380, 120], [380, 155], [40, 155]]
        }
    ]
}
```

D. How the Web Dashboard (`src/app.py`) Consumes This Output: * Takes `"annotated_image"` → displays side-by-side with original input in the right column. * Takes `"total_text_blocks"` → displays on Metric Card 1: `"Text Blocks Detected: 6"`. * Calculates average confidence → displays on Metric Card 2: `"Average Confidence: 84.5%"`. * Measures processing latency → displays on Metric Card 3: `"Processing Latency: 640 ms"`. * Takes `"detections"` → populates the Extracted Characters text box and feeds 1-click Download buttons (TXT, CSV, JSON).

Module 3: `src/app.py` (Streamlit Interactive Web Dashboard)

3.1 Layout & Controls

- **Sidebar Controls:** Active AI Model status badge (🟢 `AI Engine: Deep Learning`), Confidence Sensitivity Slider (`0.10` to `0.95`), Adaptive CLAHE toggle, and 100% On-Premise Privacy guarantee badge.
- **Tab 1 (1-Click Demos):** Instant benchmark buttons for 🚗 Vehicle Number Plate, 🧾 Store Receipt, 📄 Printed Document, and 🪧 Tilted Signboard.
- **Tab 2 (Upload Photo):** Drag & drop uploader supporting `.jpg`, `.png`, `.jpeg`, and `.bmp`.
- **Tab 3 (Live Camera):** Real-time snapshot capture via `st.camera_input`.
- **Dual-Pane Canvas:** Original image on the left, annotated bounding box image on the right.
- **Multi-Format Export Buttons:** 1-Click `Download as TXT`, `Download as CSV`, and `Download as JSON`.
- **Launch Command:** `streamlit run src/app.py`.

Module 4: `tests/` (Automated QA Test Suite)

4.1 Purpose of the Test Suite

The test suite guarantees that all utility functions, AI character recognition pipelines, edge cases, and real-world defect images function correctly before client release. It runs via: `pytest tests/ -v`.

4.2 Test Data Fixtures (`tests/test_images/`)

- `1_car_plate_motion_blur.jpg`: Vehicle license plate with horizontal motion blur (`KA 03 MG 4589`).
- `2_receipt_shadow_defect.jpg`: Thermal store receipt with harsh diagonal shadow gradient (`ASPIRIN`, `TOTAL DUE`).
- `3_document_skewed_tilted.jpg`: Business purchase order tilted by 15° (`PURCHASE ORDER #89201`).
- `4_signboard_perspective_blur.jpg`: Warning signboard with 3D perspective distortion (`ROAD WORK AHEAD`).
- `5_car_plate_glare_defect.jpg`: Vehicle license plate with bright headlight glare reflection (`DL 01 AB 78`).

4.3 Test Case Specifications

Test Function	What It Tests	Assertion Rule
`test_validate_image_bytes_valid_png()`	Upload validation with valid PNG bytes.	`assert validate_image_bytes(valid_bytes) is True`
`test_validate_image_bytes_empty()`	Rejection of 0-byte upload.	`assert validate_image_bytes(b"") is False`
`test_validate_image_bytes_corrupt()`	Rejection of random garbage bytes.	`assert validate_image_bytes(corrupt) is False`
`test_validate_image_bytes_oversized()`	Rejection of payloads > 200 MB (DoS protection).	`assert validate_image_bytes(oversized) is False`
`test_estimate_skew_angle_straight()`	Verifies flat image has near-	`assert abs(angle) < 5.0`

	zero skew angle.	
`test_preprocess_document_image_rgb()`	Contrast and bilateral filtering on 3-channel images.	`assert processed.shape[2] == 3 and not np.isnan(processed).any()`
`test_draw_ocr_bounding_boxes_clamping()`	Margin clamping for text at `(0, 0)` and `(W, H)`.	Asserts banner coordinates clamped within `(0, W)` and `(0, H)`.
`test_format_export_data_csv()`	CSV table structure and column headers.	`assert "Line_ID,Text,Confidence,Bounding_Box" in csv_str`
`test_load_ocr_engine_returns_wrapper()`	Deep learning engine initialization.	`assert engine.engine_type == "deep_learning"`
`test_disambiguate_license_plate_digits()`	Converts letter `O` to `0` and `I` to `1` in digit slots.	`assert disambiguate_license_plate("KAO3 MG4589") == "KA 03 MG 4589"`
`test_ocr_car_plate_motion_blur()`	AI accuracy on motion-blurred car plate.	`assert "KA" in text and "4589" in text`
`test_ocr_receipt_shadow_defect()`	AI accuracy through harsh diagonal shadow.	`assert "PHARMACY" in text and "TOTAL" in text`
`test_ocr_document_skewed_tilted()`	Auto-deskew	`assert "PURCHASE ORDER" in text and "89201" in text`

	and extraction on 15° tilted document.	
`test_ocr_blank_white_image()`	Edge case: blank white photo with zero text.	`assert result["total_text_blocks"] == 0` without crash

4.4 Visual Verification Artifact Generation (`output/`)

- `output/result_1_car_plate_motion_blur.jpg`: Shows `KA 03 MG 4589` enclosed in high-contrast bounding box.
- `output/result_2_receipt_shadow_defect.jpg`: Shows all line items extracted through shadow.
- `output/result_3_document_skewed_tilted.jpg`: Shows straightened purchase order with 6 lines extracted.
- `output/edge_cases/`: Visual artifacts verifying salt-and-pepper noise, wide banners, and inverted dark mode.
- `output/diverse_tests/`: Visual artifacts verifying commercial truck plates, medical prescriptions, and boarding passes.

---

4. Error Handling & Edge-Case Matrix

Edge Case Scenario	Potential Failure Point	System Defense Mechanism
Empty or 0-byte file uploaded	Crash during image decode	`validate_image_bytes()` checks file length and returns clean error message.
Corrupted / Truncated image	Unhandled decode exception	Caught in two-phase PIL validation (`verify()` + `load()`) without breaking session.
Oversized payload (> 200 MB**)**	Host memory starvation / DoS	Hard 200MB limit halts processing before memory allocation.
Photo with 0 text characters	Loop index out-of-range	Function returns `total_text_blocks = 0`, original image displayed intact with banner: *"No text detected."*
Text tilted up to 45°	OCR fragmentation / misreads	`estimate_skew_angle()` computes contour angle; `rotate_image()` counter-rotates frame straight.
Headlight glare	Blinding character	CLAHE equalizes *L*-channel histogram in LAB

on vehicle plate	washout	color space, restoring character contrast.
Text touching image boundary	Negative / out-of-bounds coordinates	Bounding badge coordinates are mathematically clamped: `startX = max(0, startX)`.

---

5. Traceability Matrix: PRD Requirements to Code

PRD Feature	Requirement Description	Implementing Function / File
Feature 1	Universal text detection on plates, receipts, documents	`src/ocr_engine.py` → `detect_and_extract_text()`
Feature 2	Vehicle license plate alphanumeric disambiguation (`0` vs `O`)	`src/ocr_engine.py` → `disambiguate_license_plate()`
Feature 3	Auto-deskew tilted text up to 45°	`src/utils.py` → `estimate_skew_angle()`, `rotate_image()`
Feature 4	CLAHE shadow and glare normalization	`src/utils.py` → `preprocess_document_image()`
Feature 5	Draw bounding boxes with boundary clamping	`src/utils.py` → `draw_ocr_bounding_boxes()`
Feature 6	1-Click pre-loaded benchmark demos	`src/app.py` → Tab 1 (`generate_synthetic_demo_image()`)
Feature 7	Custom file uploader (JPG, PNG, BMP)	`src/app.py` → Tab 2 (`st.file_uploader()`)
Feature 8	Live webcam capture	`src/app.py` → Tab 3 (`st.camera_input()`)
Feature 9	Confidence sensitivity slider	`src/app.py` → `st.sidebar.slider()`
Feature 10	Dual-pane side-by-side comparison	`src/app.py` → `st.columns(2)`
Feature 11	Multi-format export (TXT, CSV, JSON)	`src/utils.py` → `format_export_data()` & `st.download_button()`
Quality Gate	Automated verification across unit and real defect images	`tests/test_utils.py`, `tests/test_ocr_engine.py` → 26/26 PyTests